

Breast Cancer Detection

Team Contribution:

Member	ID
Nourhan Hamed Abdullah	221050219
Shahd Hussein Mohy	221050224
Mariam Adel Nassif	221050136

Introduction

This project presents the design and implementation of a Breast Cancer Analysis & Prediction System developed as an Artificial Intelligence healthcare application. The primary objective is to analyze medical tumor data and classify tumors as either Malignant or Benign using Deep Learning techniques.

To achieve intelligent healthcare prediction and analysis, the system integrates multiple Artificial Intelligence stages:

- 1. Data Preprocessing:** For cleaning healthcare data and preparing it for Neural Network training.
- 2. Feature Scaling:** For improving model learning efficiency and prediction performance.
- 3. Artificial Neural Network (ANN):** For Deep Learning healthcare classification and tumor prediction.
- 4. Accuracy and Loss Analysis:** For evaluating model learning behavior and optimization.
- 5. Prediction System:** For generating intelligent healthcare predictions using tumor measurements.

By combining Deep Learning and healthcare analysis techniques, the system transforms raw medical measurements into intelligent healthcare insights within an integrated healthcare prediction framework.

Dataset Information

- Dataset Name: Breast Cancer Wisconsin Dataset.
- Source: Kaggle.
- Link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Description: The dataset contains healthcare tumor measurements used for breast cancer classification and prediction.

Overview:

A healthcare dataset containing tumor measurement information collected from breast cancer patients and used for analysis, prediction, and healthcare classification.

Key Features:

- radius_mean: Mean radius of tumor cells.
- texture_mean: Surface texture measurement.
- perimeter_mean: Tumor perimeter measurement.
- area_mean: Tumor area measurement.

[illegible]

```
# loading the data to a data frame
data_frame = pd.DataFrame(breast_cancer_dataset.data, columns = breast_cancer_dataset.feature_names)

# print the first five rows of the dataframe
data_frame.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05967	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.19520	0.2587	0.08744	...	14.91	26.50	98.87	567.7	0.2088	0.8963	0.6869	0.2575	0.8638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678

5 rows x 30 columns

```
# adding the 'target' column to the data frame
data_frame['label'] = breast_cancer_dataset.target

# printing the last 5 rows
data_frame.tail()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	label
564	21.56	22.39	142.00	1479.0	0.11100	0.11590	0.24390	0.13890	0.1726	0.05623	...	26.40	166.10	2027.0	0.14100	0.21130	0.4107	0.2216	0.2060		0.07115	0
565	20.13	28.25	131.20	1261.0	0.09780	0.10340	0.14400	0.09791	0.1752	0.05533	...	38.25	155.00	1731.0	0.11660	0.19220	0.3215	0.1628	0.2572		0.08637	0
566	16.60	26.08	108.30	858.1	0.08455	0.10230	0.09251	0.05302	0.1580	0.05648	...	34.12	126.70	1124.0	0.11390	0.30940	0.3403	0.1418	0.2218		0.07820	0
567	20.60	20.33	140.10	1265.0	0.11780	0.27700	0.35140	0.15200	0.2387	0.07016	...	39.42	184.60	1821.0	0.16500	0.86810	0.9387	0.2650	0.4087		0.12400	0
568	7.76	24.54	47.92	181.0	0.05263	0.04362	0.00000	0.00000	0.1587	0.05884	...	30.37	58.16	268.6	0.08996	0.06444	0.0000	0.0000	0.2871		0.07039	1

5 rows x 31 columns

```
# statistical measures about the data
data_frame.describe()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	label
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000
mean	14.12722	19.28849	91.96933	654.889104	0.096360	0.104341	0.08799	0.048919	0.181162	0.062798	...	25.677223	107.281213	880.583128	0.132369	0.254285	0.272188	0.114606	0.290076	0.083846	0.074717	
std	3.524049	4.301036	24.269881	351.914129	0.014064	0.052813	0.079729	0.038803	0.027414	0.007960	...	6.146256	33.802542	569.356993	0.022832	0.157336	0.208824	0.065732	0.061867	0.018061	0.483918	
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.049960	...	12.020000	50.410000	185.200000	0.071170	0.027290	0.000000	0.000000	0.156000	0.050640	0.000000	
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064020	0.029310	0.161900	0.057700	0.161900	...	21.080000	84.110000	515.300000	0.116600	0.147200	0.116600	0.064030	0.250400	0.071400	0.000000	
50%	13.370000	18.840000	88.240000	551.100000	0.095870	0.092630	0.091540	0.033500	0.176200	0.061540	...	25.410000	87.860000	686.500000	0.131300	0.219800	0.226700	0.099930	0.282200	0.080840	1.000000	
75%	15.700000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.068120	...	29.720000	125.400000	1084.000000	0.146000	0.339100	0.362900	0.161400	0.317800	0.062880	1.000000	
max	28.110000	39.280000	188.500000	2591.000000	0.163400	0.345400	0.426000	0.291200	0.394000	0.097440	...	49.540000	251.200000	4254.000000	0.222600	1.080000	1.252000	0.291000	0.663800	0.207500	1.000000	

8 rows x 31 columns

```
# checking the distribution of target variable
data_frame['label'].value_counts()
```

label	count
1	357
0	212

dtype: int64

1 -> Benign
0 -> Malignant

Data Preprocessing and Feature Scaling

Code Explanation:

- 1. Feature Selection: The system separates healthcare measurements from target labels.
- 2. Data Splitting: The dataset is divided into training and testing datasets.
- 3. Feature Scaling: StandardScaler normalizes healthcare measurements to improve Neural Network learning performance.

Run Results Analysis:

- 1. Balanced Dataset Preparation: The dataset was successfully divided into training and testing data.
- 2. Improved Learning Efficiency: Standardization optimized Neural Network learning behavior.
- 3. Better Prediction Performance: Feature scaling improved healthcare classification accuracy.

```

1 X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state = 2)
1
1 print(X.shape, X_train.shape, X_test.shape)
1
1 (569, 30) (455, 30) (114, 30)
1
1 from sklearn.preprocessing import StandardScaler
1
1 scaler = StandardScaler()
1
1 X_train_std = scaler.fit_transform(X_train)
1
1 X_test_std = scaler.transform(X_test)
1
1 print(X_train_std)
1
1 ... [[[-0.01330339  1.7757658  -0.01491962 ... -0.13236958 -1.08014517
1      -0.03527943]
1      [-0.8448276  -0.6284278  -0.87702746 ... -1.11552632 -0.85773964
1      -0.72098905]
1      [ 1.44755936  0.71180168  1.47428816 ...  0.87583964  0.4967602
1      0.46321706]
1      ...
1      [-0.46608541 -1.49375484 -0.53234924 ... -1.32388956 -1.02997851
1      -0.75145272]
1      [-0.50025764 -1.62161319 -0.527814 ... -0.0987626  0.35796577
1      -0.43906159]
1      [ 0.96060511  1.21181916  1.00427242 ...  0.8956983  -1.23064515
1      0.50697397]]

```

Artificial Neural Network (ANN)

Code Explanation:

1. Neural Network Initialization: The ANN model was created using TensorFlow and Keras.
2. Input Layer: The model accepts healthcare tumor measurements as input features.
3. Hidden Layer: Dense layers with ReLU activation extract healthcare patterns.
4. Output Layer: The ANN predicts tumor classification probabilities.
5. Compilation: Adam optimizer and sparse categorical crossentropy loss were used during model compilation.

Run Results Analysis:

1. Deep Learning Capability: The ANN successfully learned healthcare tumor patterns.
2. Medical Pattern Recognition: Hidden layers extracted important healthcare relationships.
3. Healthcare Prediction: The model demonstrated strong breast cancer classification capability.

```

# importing tensorflow and keras
import tensorflow as tf
tf.random.set_seed(3)
from tensorflow import keras

model = keras.Sequential([
    keras.Input(shape=(30,)),
    keras.layers.Flatten(),
    keras.layers.Dense(20, activation='relu'),
    keras.layers.Dense(2, activation='sigmoid')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])

```

Neural Network Training

Code Explanation:

1. Model Training: The ANN was trained using standardized healthcare measurements.
2. Validation Analysis: Validation data was used to monitor model learning performance.
3. Epoch Monitoring: The system tracked accuracy and loss during training.

Run Results Analysis:

1. Accuracy Improvement: Model accuracy gradually improved during training.

2. Loss Reduction: Training loss decreased over multiple epochs.
3. Stable Learning Process: Validation accuracy demonstrated successful ANN learning behavior.

```
# training the neural network
history = model.fit(X_train_std, Y_train, validation_split=0.1, epochs=10)
```

Epoch	1/10	2/10	3/10	4/10	5/10	6/10	7/10	8/10	9/10	10/10
13/13	2s 40ms/step	0s 14ms/step	0s 19ms/step	0s 14ms/step	0s 16ms/step	0s 17ms/step	0s 12ms/step	0s 11ms/step	0s 9ms/step	0s 11ms/step
accuracy	0.4792	0.6528	0.7800	0.8704	0.8924	0.9218	0.9364	0.9438	0.9462	0.9487
loss	0.9385	0.6404	0.4575	0.3472	0.2793	0.2351	0.2048	0.1828	0.1661	0.1528
val_accuracy	0.6522	0.7391	0.8043	0.8696	0.8696	0.9130	0.9130	0.9348	0.9348	0.9348
val_loss	0.6562	0.4676	0.3611	0.2976	0.2561	0.2262	0.2034	0.1853	0.1707	0.1586

Accuracy Analysis

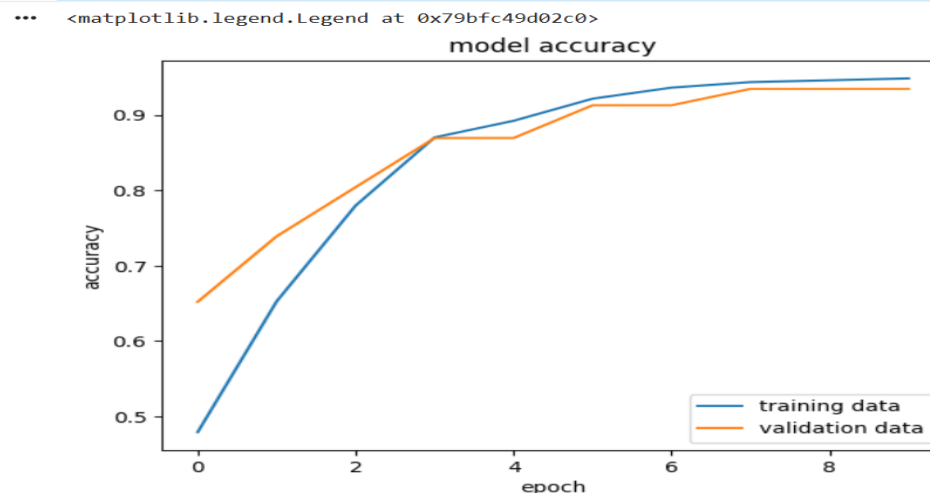
Code Explanation:

1. Accuracy Monitoring: The system visualized training accuracy.
2. Validation Comparison: Validation accuracy was compared with training accuracy.
3. Learning Visualization: Graphs demonstrated Neural Network learning behavior.

Run Results Analysis:

1. High Classification Accuracy: The ANN achieved strong healthcare prediction performance.
2. Stable Validation Accuracy: Validation accuracy remained close to training accuracy.
3. Effective Learning: The model successfully generalized healthcare patterns.

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['training data', 'validation data'], loc = 'lower right')
```



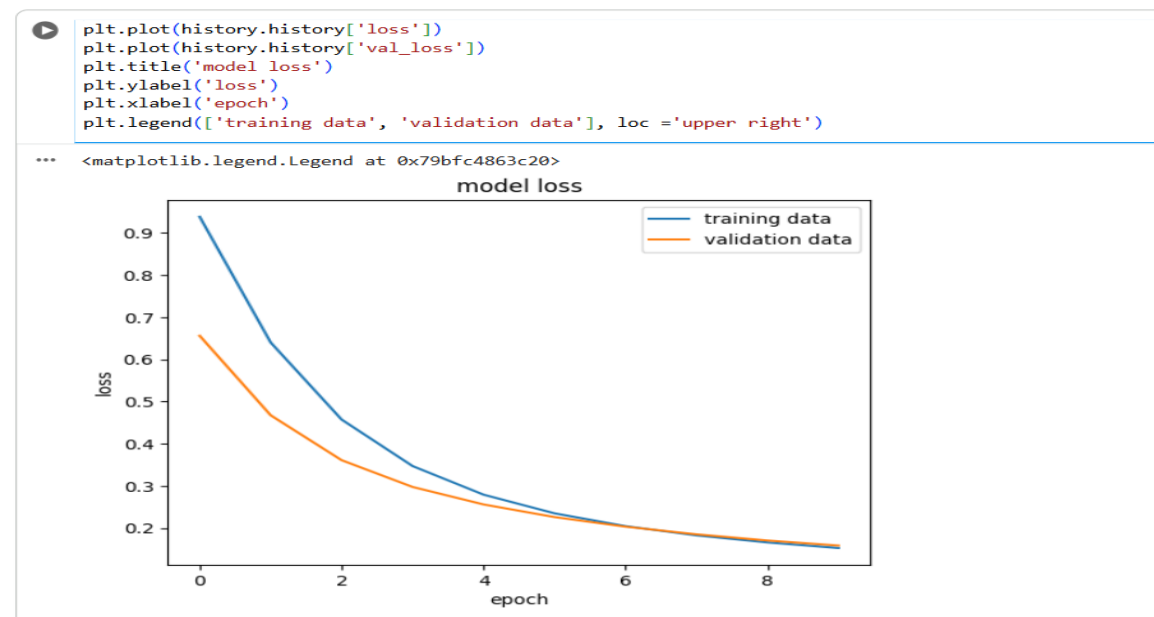
Loss Analysis

Code Explanation:

1. Loss Monitoring: The system tracked training and validation loss.
2. Performance Evaluation: Lower loss values indicate improved prediction performance.
3. Convergence Analysis: Loss graphs help visualize Neural Network optimization.

Run Results Analysis:

1. Reduced Prediction Error: Loss gradually decreased during training.
2. ANN Stability: The model became more stable during learning.
3. Model Optimization: The optimizer improved healthcare prediction capability.



Model Evaluation

Code Explanation:

1. Model Testing: The ANN was evaluated using unseen testing data.
2. Accuracy Calculation: The system measured healthcare classification performance.
3. Performance Validation: Testing accuracy verified model reliability.

Run Results Analysis:

1. Accurate Classification: The ANN achieved high healthcare classification accuracy.
2. Reliable Prediction Capability: The model successfully classified tumor conditions.
3. Generalization Performance: The ANN performed well on unseen healthcare data.

```
loss, accuracy = model.evaluate(X_test_std, Y_test)
print(accuracy)
```

4/4 ————— 0s 16ms/step - accuracy: 0.9561 - loss: 0.1571
0.9561403393745422

Breast Cancer Prediction System

Code Explanation:

1. Input Processing: Healthcare measurements are converted into NumPy arrays.
2. Standardization: Input values are normalized using the trained scaler.
3. Prediction Generation: The ANN predicts tumor classification.
4. Result Interpretation: The system displays whether the tumor is Malignant or Benign.

Run Results Analysis:

1. Intelligent Healthcare Prediction: The ANN generated healthcare predictions successfully.
2. Medical Decision Support: The system supports healthcare diagnosis analysis.
3. Real-World Healthcare Application: The model demonstrates Artificial Intelligence usage in healthcare systems.

```
input_data = (17.99,10.38,122.0,1001,0.1184,0.2776,0.3801,0.1471,0.2419,0.07871,1.095,0.9053,0.589,153.4,0.006399,0.04904,0.05373,0.01587,0.03003,0.006193,25.38,17.33,184.6,2019,0.1622,0.6656,0.7119,0.2654,0.4601,0.1189)
input_data_as_numpy_array = np.asarray(input_data)
input_data_reshape = input_data_as_numpy_array.reshape(1,-1)
input_data_std = scaler.transform(input_data_reshape)
prediction = model.predict(input_data_std)
print(prediction)
prediction_label = [np.argmax(prediction)]
print(prediction_label)
if(prediction_label[0] == 0):
    print('Tumor is Malignant')
else:
    print('Tumor is Benign')
```

```
1/1 ----- 0s 58ms/step
[[0.7882364  0.00758667]]
[0.1654(0)]
Tumor is Malignant
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but StandardScaler was fitted with feature names
  warnings.warn(msg)
```

System Integration

The project combines multiple Artificial Intelligence and Deep Learning stages into a single integrated healthcare prediction system.

- Data Preprocessing → Healthcare data preparation.
- Feature Scaling → Neural Network optimization.
- Artificial Neural Network → Deep Learning healthcare classification.
- Accuracy and Loss Analysis → Model performance evaluation.
- Prediction System → Intelligent breast cancer diagnosis support.

This integration demonstrates how multiple Artificial Intelligence approaches can work together within an intelligent healthcare expert system.

Conclusion

This project successfully implemented an Artificial Intelligence Breast Cancer Analysis & Prediction System using a real-world healthcare dataset.

Data preprocessing and feature scaling were used to prepare healthcare measurements for Deep Learning analysis. Artificial Neural Networks were used for healthcare classification and intelligent tumor prediction.

The project demonstrated how Deep Learning techniques can transform healthcare measurements into intelligent medical insights that support healthcare understanding and medical decision-making.